Guion de exposición · Proyecto Integrador 1

Minería de Datos Masivos y Flujos de Datos · César Pajuelo y Aleksander Paico · 11 de octubre de 2026

Reparto y tiempos

La exposición dura unos 20 minutos para 20 diapositivas: César abre con los datos, el EDA y la similitud (Partes I y II); Aleksander sigue con ANN, flujos, reglas y la discusión (Partes III a VI). La demostración la hacen entre los dos. Si el tiempo asignado es menor, recorten primero las diapositivas 4, 6 y 17: su contenido está en el reporte web.

Diap.	Tema	Expone	Tiempo
1	Portada	César	0:40
2	Tres preguntas de interés	César	1:00
3	Por qué OCDS	César	1:15
4	Proceso KDD	César	1:15
5	Quiebre de 2015	César	1:30
6	Concentración del dinero	César	1:00
7	RDD vs DataFrames y particionado	César	1:30
8	MinHash	César	1:30
9	LSH: bandas y filas	César	1:30
10	LSH a escala (pasa la palabra)	César	1:30
11	IVF y HNSW	Aleksander	2:00
12	La pérdida está en LSA; respuesta a P2	Aleksander	1:30
13	Bloom, Count-Min, DGIM	Aleksander	2:00
14	Apriori vs FP-Growth	Aleksander	1:30
15	Camarillas y registros pasivos	Aleksander	2:00
16	¿Vale la pena aproximar?	Aleksander	1:30
17	Spark y escala 10×/100×	Aleksander	1:30
18	Ética	César	1:00
19	Demo: web y código	Ambos	2:00
20	Respuestas y cierre	César	1:00

Cada diapositiva lleva el mismo texto en las notas del orador del archivo `presentacion/Presentacion_Proyecto_Integrador.pptx` (vista Moderador).

Guion de César (diapositivas 1-10 y 18)

1 · Portada. "Buenos días. Somos César Pajuelo y Aleksander Paico. Aplicamos las seis familias de técnicas del curso al registro completo de contrataciones públicas del Perú: 2,76 millones de procesos del SEACE, publicados por el OECE en estándar OCDS. El título resume nuestras tres preguntas: quién compite, quién se repite y cómo vigilarlo. Yo presento los datos, el EDA en Spark y la similitud; Aleksander, vecinos aproximados, flujos, reglas y la discusión. Al final mostramos el reporte web y el código en vivo."

2 · Tres preguntas. "Formulamos las preguntas después de perfilar el dataset, para que se pudieran responder con los campos reales. Ninguna es descriptiva. P1: si un grupo de empresas se presenta siempre en bloque, la competencia puede ser solo aparente. P2: reutilizar especificaciones puede ser estandarización sana o términos de referencia copiados; cruzamos la similitud del texto con quién gana. P3: un órgano de control no puede reprocesar el histórico cada día; ¿cuánta precisión perdemos vigilando con resúmenes? Cada pregunta usa al menos dos técnicas, y eso es lo que integra el proyecto."

3 · Por qué OCDS. "Evaluamos las seis fuentes sugeridas. OCDS es la única que cumple los cuatro requisitos de forma natural: volumen, 17 GB en CSV y tablas de hasta 12,8 millones de filas; conjuntos comparables, texto por proceso y RUC por postor; una fecha al minuto sin nulos; y una canasta densa, proceso→postores. Descargamos las 25 particiones con un script automatizado desde el OCP Data Registry, con licencia CC BY 4.0, y verificamos fila a fila los conteos contra lo que declara el publicador."

4 · KDD. "Me quedo con lo que no era obvio. main.csv tiene cinco esquemas distintos según el año: un glob desalinea columnas en silencio, por eso leemos partición por partición y unimos con unionByName. Excluimos 52 603 procesos con monto ≤ 0 y recalculamos el número de postores. Particionamos el Parquet por año de publicación, no por el año del archivo, porque no coinciden. Y materializamos una sola vez las representaciones que reutilizan las Partes II, III y V."

5 · Quiebre de 2015. "Este hallazgo cambió todo el proyecto. Entre 2004 y 2014 hay de 1,5 a 2,3 postores por proceso; en 2015 la media salta a 4 y llega a 14,6 en 2024. Un cambio así en un año no describe un mercado que se volvió competitivo: describe un cambio en lo que el sistema registra. Antes se publicaba sobre todo al ganador. Tomarlo literal daría el hallazgo espectacular, y falso, de que la competencia se multiplicó por siete. Decisión: toda pregunta de competencia se resuelve sobre 2015-2026."

6 · Dinero concentrado. "Tres datos de distribución. El 1 % de las 3 163 entidades concentra el 40,6 % del monto: proyectos del MTC, EsSalud, Sedapal. Los procesos de diez millones o más son el 1,6 % pero mueven el 58 % del dinero. Y la licitación pública tiene siete veces menos procesos que la adjudicación simplificada pero 2,6 veces más dinero. Para P3 esto es un caso de heavy hitters: pocos actores explican casi todo."

7 · RDD vs DataFrames. "Implementamos dos agregaciones de las preguntas en RDD con map y reduceByKey, en DataFrames y en Spark SQL. El resultado es el mismo, pero el RDD tarda 22,6 s contra 0,39 s. En PySpark cada fila del RDD viaja de la JVM a Python y vuelve; el DataFrame se queda en la JVM y Catalyst agrega antes del shuffle. SQL es el más legible; el RDD lo reservamos para el Apriori. Particionado: al filtrar por año, el plan muestra PartitionFilters y Spark lee 1 de 25 directorios: de 0,61 a 0,15 s."

8 · MinHash. "Cada proceso es el conjunto de sus shingles de 5 caracteres. Primero la verdad: Jaccard exacto sobre 2 000 procesos, 22 segundos. Extrapolado a 883 mil procesos son 49 días, porque crece con n^2. MinHash propio: 256 funciones hash escritas en Spark SQL, sin UDF de Python. El error sigue $1/\sqrt{T}$ y queda bajo la cota teórica. Elegimos T = 128: RMSE 0,02, que no cambia qué pares superan 0,8."

9 · LSH. "Partimos la firma en b bandas de r filas. La tabla es la curva S vista desde los datos: con r = 2 recuperamos todo pero con 577 mil falsos positivos; con r ≥ 16, precisión perfecta pero perdemos casi todos. Elegimos b = 32, r = 8: umbral 0,65, 85 % de recall y solo 37 candidatos, que un filtro exacto limpia. MinHashLSH de MLlib usa siempre r = 1, así que su umbral es 1/b: volcó más de 11 GB a disco y no terminó."

10 · LSH a escala. "A escala salen 589 mil pares con Jaccard ≥ 0,8, y el 98 % son de la misma entidad: no es plagio, son reconvocatorias. LSH terminó siendo una herramienta de limpieza: la Parte V las colapsa. Para P1, cada proveedor es el conjunto de sus procesos: 2 275 pares acoplados y un grupo de cuatro que aparece en exactamente los mismos 37 procesos. Puede ser un consorcio: es una pista, no una conclusión. Le paso la palabra a Aleksander."

18 · Ética. "Con datos abiertos, cualquiera puede priorizar qué revisar entre millones de procesos. Pero un RUC que empieza con 10 es una persona natural, y un falso positivo puede tener una explicación legítima. Por eso publicamos etiquetas anónimas y siempre la lectura alternativa. Y el quiebre de 2015 y los registros pasivos muestran que un dato público mal entendido produce conclusiones falsas."

Guion de Aleksander (diapositivas 11-17)

11 · IVF y HNSW. "Calculamos TF-IDF en Spark SQL sobre 885 mil procesos y lo reducimos con LSA a 256 dimensiones, porque FAISS necesita vectores densos y el TF-IDF denso pesaría 70 GB. IVF agrupa los vectores con k-means en nlist celdas, y la consulta solo recorre las nprobe más cercanas. HNSW es un grafo en capas: arriba, saltos largos; abajo, todos los nodos; la búsqueda es voraz y su costo, logarítmico. La búsqueda exacta tarda 57,6 ms. IVF con nprobe 16 llega a recall 0,98 en 0,48 ms, 120 veces más rápido. HNSW gana a latencia muy baja, 0,95 en 0,14 ms, pero se estanca en 0,98: los miles de vectores idénticos de las reconvocatorias crean mesetas donde la búsqueda voraz se atasca. Elegimos IVF."

12 · La pérdida está en LSA. "Este es el hallazgo más importante de la Parte III. Comparado con el TF-IDF exacto, LSA con 256 dimensiones solo recupera el 33 % de los 10 vecinos reales. Las descripciones son cortas y el vocabulario enorme. Ningún índice recupera lo que la SVD ya perdió, así que P2 la respondimos con vecinos exactos del TF-IDF disperso: 13 ms por consulta. Respuesta: dos procesos al azar comparten ganador el 0,02 % de las veces; con texto casi idéntico, el 15 %; entre entidades distintas, todavía el 10 %. Puede ser un mercado especializado o TDR copiados: sirve para priorizar auditorías."

13 · Flujos. "Spark ordena el flujo por fecha y calcula la verdad exacta; las tres estructuras las implementamos desde cero, en una sola pasada. Bloom responde '¿este RUC ya postuló?'. No tiene falsos negativos; el error es marcar como ya visto a un proveedor nuevo. Con 8 bits por clave ocupa 0,67 MB, 100 veces menos que el set, y pierde el 0,41 % de las alertas. Count-Min, con 512 KB frente a 86 MB, recupera todo el top 20, pero en la cola larga convierte un 1 en 153: responde quién domina, no cuánto postuló un proveedor pequeño. DGIM cuenta los procesos de más de un millón del último mes con 4 a 11 % de error y el 2 % de la memoria. Siendo honestos: a este volumen la ventana exacta de un año pesa 6 KB."

14 · Apriori vs FP-Growth. "Cada proceso es una canasta y los ítems son los RUC postores. Colapsamos 35 100 reconvocatorias y quitamos las canastas de 1 postor o de más de 40, que son compras masivas. Quedan 618 mil. Apriori propio sobre RDD y FP-Growth de MLlib dan exactamente los mismos conjuntos y soportes: eso valida la implementación. Apriori tardó 484 s para llegar a tamaño 4; FP-Growth, 26 s y sin tope. Hay 585 mil conjuntos frecuentes porque un grupo de 14 RUC genera 2^14 subconjuntos; por eso analizamos los 183 mil maximales."

15 · Camarillas y registros pasivos. "'Gana el grupo' no dice nada solo: si 11 de 12 postores son del grupo, ganar el 92 % es lo esperable. Por eso lo comparamos con el azar. Patrón 1: 157 de las 300 camarillas mayores son grupos de 9 a 11 proveedores en 15 a 31 sedes del Poder Judicial, que ganan en mediana el 83 % frente al 36 % esperado. Patrón 2: P0205→P0206, confianza 0,81, 4 406 procesos en 913 entidades, y no ganan nunca. Eso no es competencia: la tabla de postores incluye registrados que no ofertan. Es un hallazgo de calidad de datos, como el de 2015."

16 · ¿Vale la pena aproximar? "En valor: EDA, LSH y FP-Growth, alto; Bloom y Count-Min, medio, porque hoy el conjunto exacto cabe en memoria; ANN, medio-bajo; DGIM, bajo a esta escala; Apriori, didáctico. La lección: la aproximación del algoritmo casi nunca fue el problema; la de la representación sí. LSA perdió dos tercios de los vecinos, más que cualquier índice. Medir contra la verdad exacta en cada paso fue lo que nos permitió verlo."

17 · Spark y escala. "Spark valió la pena en la ingesta, los joins de 10 millones de filas, el TF-IDF, el LSH a escala y FP-Growth. En los sketches no hacía falta, porque son secuenciales por definición, ni en FAISS. El overhead se notó: RDD 58 veces más lento y 37 minutos para las firmas MinHash. Con 10× o 100× el volumen, la fuerza bruta pasa a 13 o 1 300 años, HNSW deja de caber en una máquina y habría que usar IVF-PQ. Los sketches casi no crecen: a 100×, DGIM pasa de curiosidad a necesidad. Le devuelvo la palabra a César."

Demostración en vivo y cierre (diapositivas 19-20)

Dejen abiertos antes de empezar el reporte (https://cesarpr30.github.io/Proyecto_DataMining/) y VS Code con los tres scripts; si falla internet, abran `web/index.html` en local (los gráficos necesitan el CDN de Chart.js, así que lleven capturas de respaldo).

César maneja el reporte web (≈ 1 min).

1. Parte II, curva S: pasar de b = 32, r = 8 a b = 16, r = 16 y mostrar que el umbral sube a 0,84 y el recall cae al 10 %.
2. Parte IV, Bloom: bajar de 6 a 4 bits por clave y mostrar que el error salta del 1,4 % al 4,6 %.
3. Parte V, reglas: subir la confianza a 80 % y ver cuántas reglas quedan.
4. Decir: "Spark corre una sola vez offline; la página lee artefactos ligeros y solo recalcula en vivo las fórmulas de los compromisos."

Aleksander muestra el código (≈ 1 min).

1. `scripts/10_parte2_similitud.py`: el MinHash con funciones de orden superior de Spark SQL (sin UDF) y por qué el primo 2^31−1.
2. `scripts/14_parte5_reglas.py`: el Apriori por niveles con flatMap + reduceByKey y la poda con broadcast.
3. `scripts/13_parte4_flujos.py`: el Bloom propio con doble hashing.
4. Decir: "Cada script indica en su cabecera qué corre en Spark y qué en local, y todo se reproduce con ejecutar_todo.py."

20 · Respuestas (César). "Cerramos con las respuestas. P1: sí, en dos escalas; la red del Poder Judicial gana el 83 % frente al 36 % esperado. P2: el 98 % de los casi-duplicados son reconvocatorias, pero el texto casi idéntico anticipa al ganador: 15 % frente a 0,02 %. P3: sí se puede vigilar con memoria acotada: un Bloom 100 veces más chico pierde el 0,41 % de las alertas. La conclusión: lo aproximado casi nunca fue el problema; la representación y la calidad del dato sí. Gracias; quedamos atentos a sus preguntas."

Preguntas probables y respuestas

Responde quien expuso esa parte; el otro completa si hace falta. Respuestas de 20-30 segundos: primero la cifra, después el porqué.

Datos y Parte I (César)

- **¿Cómo saben que el salto de 2015 es del registro y no del mercado?** Es un salto de 2,3 a 4 postores en un solo año, que sigue subiendo hasta 14,6, y el postor único cae del 56 % al 32 %. Ningún cambio de mercado es tan brusco; coincide con el cambio en cómo el SEACE publica a los participantes. Por prudencia analizamos solo 2015-2026.
- **¿Realmente necesitaban Spark para 17 GB?** Para la ingesta y los joins sí: postores (10 M filas) junto con partes (12,8 M) no caben cómodos en pandas con 24 GB. Para los sketches y FAISS no, y por eso esa parte corre en local.
- **¿Por qué el RDD es 58 veces más lento?** No por ser RDD, sino por PySpark: cada fila se serializa JVM→Python→JVM. El DataFrame no sale de la JVM y Catalyst agrega antes del shuffle. En Scala la brecha sería mucho menor.
- **¿Por qué particionar por año y no por mes o entidad?** El año da 25 directorios de tamaño razonable y coincide con la ventana de análisis. Por entidad serían 3 163 directorios pequeños. A 100× particionaríamos por año y mes.
- **¿Cómo trataron los duplicados?** Ningún OCID se repite entre particiones; igual dejamos una deduplicación por OCID. Las reconvocatorias (otro OCID, mismo texto) las detectó LSH y se colapsan en la Parte V.

Parte II (César)

- **¿Por qué shingles de 5 caracteres y no palabras?** El texto mezcla códigos, siglas y redacción libre; los shingles de caracteres toleran variaciones pequeñas. Con 5 caracteres hay 174 shingles por proceso de media, suficiente para separar documentos.
- **¿Por qué implementaron LSH en vez de usar MLlib?** MinHashLSH usa r = 1, así que su umbral es 1/b: con 8 tablas, cualquier par con Jaccard 0,125 es candidato. Volcó más de 11 GB y no terminó; la nuestra con r = 8 no volcó nada.
- **Con solo 20 pares reales, ¿es fiable el recall de 85 %?** La tendencia sí, el decimal no: cada par vale 5 puntos. Por eso además mostramos la curva S teórica, que coincide con lo medido.
- **¿Cómo eligieron b y r?** Buscando que el umbral efectivo (1/b)^(1/r) quedara cerca del umbral de interés y que hubiera pocos candidatos. b = 32, r = 8 da 0,65 con 37 candidatos; los falsos positivos se eliminan con un Jaccard exacto.
- **¿Por qué el primo 2^31−1 y no 2^61−1?** Con 2^61−1, el producto a·x desborda el entero de 64 bits de Spark y el hash deja de ser uniforme.
- **¿El grupo de cuatro es colusión?** No lo afirmamos. Puede ser un consorcio, empresas vinculadas o un mercado local muy pequeño. Es una pista que justifica revisar el expediente.

Parte III (Aleksander)

- **¿Qué diferencia hay entre IVF y HNSW?** IVF particiona con k-means y busca solo en nprobe celdas; se construye rápido y ocupa lo mismo que los datos. HNSW es un grafo navegable en capas; es más rápido a recall moderado, pero ocupa más memoria y tarda más en construirse.
- **¿Por qué HNSW se estanca en 0,98?** Hay miles de vectores idénticos por las reconvocatorias: el grafo gasta aristas en duplicados y la búsqueda voraz se queda en mesetas de similitud 1,0. Es nuestra hipótesis más probable.
- **¿Por qué un recall tolerante a empates?** Si una consulta tiene 30 vecinos con coseno 1,0, cualquier subconjunto de 10 es exacto. Contar solo los mismos IDs castigaría respuestas correctas.
- **Si LSA pierde tanto, ¿por qué no usar otra representación?** Es la siguiente mejora: embeddings de un modelo de lenguaje en español, o índices sobre vectores dispersos. Lo importante es que lo medimos y respondimos P2 con el TF-IDF exacto.
- **¿Por qué excluyeron el título?** Es la nomenclatura del procedimiento ("DIRECTA-PROC-7-2016-UNT-1"): sus siglas hacían que dos compras sin relación de la misma universidad salieran con coseno 1,0.

Parte IV (Aleksander)

- **¿Cómo simularon el flujo?** Spark ordena los procesos y postulaciones por tender_datePublished; las estructuras los consumen uno a uno en ese orden, en una sola pasada.
- **¿Por qué la tasa de falsos positivos medida es menor que la teórica?** La fórmula describe el filtro lleno; en el flujo cada consulta se hace con el filtro a medio llenar.
- **Si el Bloom es más lento que un set, ¿para qué sirve?** Ahorra memoria, no tiempo: 0,67 MB frente a 69,6 MB. En Python puro es más lento; en C o en un sistema de streaming sería competitivo.
- **¿Cómo eligieron w y d en Count-Min?** El error es ≤ εN con ε = e/w y probabilidad 1 − δ con δ = e^−d. A igual memoria, una fila ancha da menos error medio; más filas reducen la probabilidad de un error grande.
- **¿DGIM tiene sentido aquí?** A este volumen no: la ventana exacta de un año pesa 6 KB. Lo dijimos así en el reporte; su valor aparece con millones de eventos al día.
- **¿Las fórmulas vectorizadas no hacen trampa?** Las validamos: dan exactamente los mismos falsos positivos que la pasada secuencial (2 900 en ambas, por ejemplo). Las usamos solo para barrer parámetros rápido.

Parte V (Aleksander)

- **¿Por qué quitar las canastas de más de 40 postores?** Son compras masivas donde coincidir no es una elección, y una canasta de 300 postores genera 45 000 pares que dominarían el conteo.
- **¿Por qué el lift no sirve para ordenar?** Cada RUC aparece en una fracción mínima de las canastas, así que todas las reglas tienen lift entre 25 y 1 700. Lo que separa son la confianza y los procesos en común.
- **¿Cómo calcularon la tasa esperada al azar?** Para cada proceso, tamaño del grupo entre número de postores, promediado sobre los procesos adjudicados.
- **¿Cómo saben que hay registros pasivos?** Parejas presentes en miles de procesos de cientos de entidades que no ganan ninguno. Nadie compite 4 406 veces sin ganar; lo más probable es que se registren para recibir información. Confirmarlo exige las ofertas evaluadas, que esta fuente no publica.

- **¿Por qué LSH y FP-Growth casi no coinciden?** Miden escalas distintas: 127 de los 162 pares de LSH tienen menos de 30 procesos, bajo el soporte de FP-Growth, y las camarillas grandes diluyen el Jaccard por pares. Son complementarias.

Discusión, ética y reproducibilidad (cualquiera)

- **¿Qué técnica aportó más?** El EDA, porque el quiebre de 2015 cambió todo el diseño; después, LSH y FP-Growth, que respondieron P1 y revelaron problemas de calidad de datos.
- **¿Qué harían con más tiempo?** Cruzar con montos adjudicados y precios para distinguir colusión de especialización, usar mejores embeddings para P2 y generar las firmas MinHash difundiendo los coeficientes.
- **¿Cómo evitan acusar a alguien?** Etiquetas anónimas (P0001), los RUC solo en artefactos locales _PRIVADO, la lectura alternativa en cada hallazgo y ningún calificativo de conducta.
- **¿Cómo se reproduce todo?** descargar_datos.py y luego ejecutar_todo.py; unos 90 minutos en un portátil de 12 núcleos. La web ya trae los artefactos y se abre sin servidor.
- **¿Qué parte hizo cada uno?** Respondan con la verdad y con el detalle de lo que cada uno implementó. El enunciado advierte que pueden pedir sustentación oral de cualquier parte, así que los dos deben poder explicar todo el pipeline.